

LAPORAN PRAKTIKUM STRUKTUR DATA

SORTING

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur penulis ucapkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan Laporan Praktikum Struktur Data materi Sorting dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu tugas praktikum pada mata kuliah Struktur Data untuk memahami konsep dasar pengurutan data menggunakan bahasa pemrograman Java. Dalam praktikum ini, penulis mempelajari beberapa metode sorting seperti Bubble Sort, Selection Sort, dan Insertion Sort. Selain membuat program sorting sederhana, penulis juga mempelajari implementasi GUI (Graphical User Interface) menggunakan Java Swing sehingga proses pengurutan data dapat ditampilkan secara visual dan langkah demi langkah.

Penulis menyadari bahwa laporan praktikum ini masih memiliki kekurangan baik dari segi isi maupun penulisan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini menjadi lebih baik di masa yang akan datang. Semoga laporan praktikum ini dapat bermanfaat bagi penulis maupun pembaca dalam memahami materi sorting pada Struktur Data.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	2
2.1 InsertionSort_2511531017.....	2
2.2 SelectionSort_2511531017.....	4
2.3 BubleSort_2511531017.....	5
2.4 InsertionSortGUI_2511531017.....	7
TUGAS.....	17
2.5 Mahasiswa_2511531017.....	17
2.6 MahasiswaGUI_2511531017.....	18
BAB III KESIMPULAN.....	32
3.1 Kesimpulan.....	32
3.2 Saran.....	32
DAFTAR PUSTAKA.....	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini membuat pengolahan data menjadi hal yang sangat penting dalam dunia pemrograman. Salah satu proses dasar dalam pengolahan data adalah sorting atau pengurutan data. Sorting digunakan untuk menyusun data berdasarkan urutan tertentu seperti dari nilai terkecil ke terbesar atau berdasarkan urutan alfabet sehingga data menjadi lebih mudah dicari dan dikelola.

Dalam struktur data terdapat beberapa algoritma sorting yang sering digunakan, di antaranya Bubble Sort, Selection Sort, dan Insertion Sort. Setiap algoritma memiliki cara kerja yang berbeda dalam melakukan proses pengurutan data. Oleh karena itu, pemahaman mengenai algoritma sorting sangat penting bagi mahasiswa agar mampu memilih metode pengurutan yang sesuai dengan kebutuhan program.

Pada praktikum ini dilakukan implementasi algoritma sorting menggunakan bahasa pemrograman Java. Selain membuat program sorting sederhana, praktikum ini juga menerapkan GUI (Graphical User Interface) menggunakan Java Swing sehingga proses pengurutan data dapat ditampilkan secara visual dan langkah demi langkah. Dengan adanya visualisasi tersebut, proses kerja algoritma sorting dapat lebih mudah dipahami.

1.2 Tujuan

1. Memahami konsep dasar sorting pada struktur data.
2. Mempelajari dan mengimplementasikan algoritma Bubble Sort, Selection Sort, dan Insertion Sort.
3. Membuat program sorting menggunakan Java dengan tampilan GUI (Java Swing).

1.3 Manfaat

1. Menambah pemahaman tentang cara kerja algoritma sorting dalam struktur data.
2. Melatih kemampuan pemrograman Java dalam pengolahan dan pengurutan data.
3. Membantu memahami proses sorting secara visual melalui aplikasi GUI yang interaktif.

BAB II

PEMBAHASAN

2.1 InsertionSort_2511531017

```
package pekan7_2511531017;

public class InsertionSort_2511531017 {
    public static void insertionSort_2511531017(int[] arr_1017) {
        int n_1017 = arr_1017.length;
        for(int i_1017 = 1; i_1017 < n_1017; i_1017++) {
            int key_1017 = arr_1017[i_1017];
            int j_1017 = i_1017 - 1;
            while(j_1017 >= 0 && arr_1017[j_1017] > key_1017) {
                arr_1017[j_1017 + 1] = arr_1017[j_1017];
                j_1017--;
            }
            arr_1017[j_1017 + 1] = key_1017;
        }
    }

    public static void main(String[] args) {
        int arr_1017[] = {23, 78, 45, 8, 32, 56, 1};
        int n_1017 = arr_1017.length;
        System.out.printf("array yang belum terurut:\n");
        for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
            System.out.print(arr_1017[i_1017] + " ");
        System.out.println("");
        insertionSort_2511531017(arr_1017);
        System.out.printf("array yang terurut:\n");
        for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
            System.out.print(arr_1017[i_1017] + " ");
        System.out.println("");
    }
}
```

Kode 2.1

Penjelasan:

```
public static void insertionSort_2511531017(int[] arr_1017) {
    int n_1017 = arr_1017.length;
    for(int i_1017 = 1; i_1017 < n_1017; i_1017++) {
        int key_1017 = arr_1017[i_1017];
        int j_1017 = i_1017 - 1;
```

Digunakan untuk memulai proses pengurutan data menggunakan algoritma Insertion Sort. Variabel `n_1017` digunakan untuk menyimpan panjang array, sedangkan perulangan `for` digunakan untuk mengambil elemen array mulai dari indeks kedua hingga elemen terakhir. Nilai array yang sedang diproses disimpan pada variabel `key_1017`, lalu variabel `j_1017` digunakan untuk menunjuk indeks sebelum elemen tersebut agar dapat dibandingkan dengan data sebelumnya.

```
while(j_1017 >= 0 && arr_1017[j_1017] > key_1017) {
    arr_1017[j_1017 + 1] = arr_1017[j_1017];
    j_1017--;
```

Melakukan proses pergeseran elemen array. Selama nilai pada indeks sebelumnya lebih besar dari key_1017, maka elemen tersebut akan digeser satu langkah ke kanan. Variabel j_1017 akan terus dikurangi untuk membandingkan key_1017 dengan elemen-elemen sebelumnya hingga ditemukan posisi yang sesuai.

```
arr_1017[j_1017 + 1] = key_1017;
```

Kode ini digunakan untuk menempatkan nilai key_1017 pada posisi yang benar setelah proses pergeseran elemen selesai dilakukan sehingga bagian array menjadi terurut.

```
public static void main(String[] args) {  
    int arr_1017[] = {23, 78, 45, 8, 32, 56, 1};  
    int n_1017 = arr_1017.length;
```

Untuk membuat method utama program dan mendeklarasikan array yang akan diurutkan. Variabel n_1017 digunakan untuk menyimpan jumlah elemen pada array.

```
System.out.printf("array yang belum terurut:\n");  
for(int i_1017 = 0; i_1017 < n_1017; i_1017++)  
    System.out.print(arr_1017[i_1017] + " ");  
System.out.println("");
```

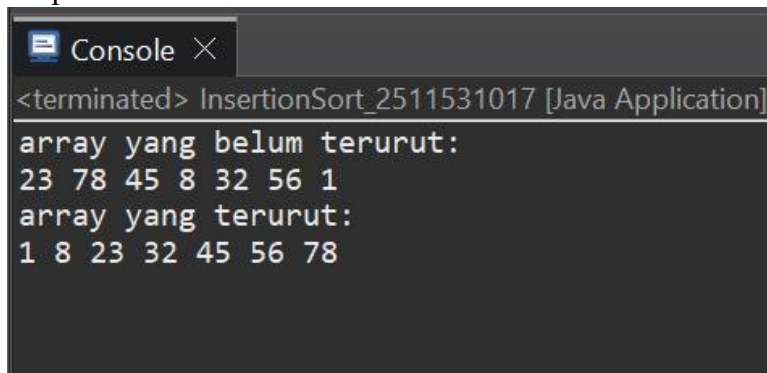
Digunakan untuk menampilkan isi array sebelum dilakukan proses sorting. Program melakukan perulangan untuk mencetak seluruh elemen array ke layar.

```
System.out.printf("array yang terurut:\n");  
for(int i_1017 = 0; i_1017 < n_1017; i_1017++)  
    System.out.print(arr_1017[i_1017] + " ");  
System.out.println("");
```

Menampilkan isi array setelah proses pengurutan selesai dilakukan. Program akan mencetak seluruh elemen array yang sudah tersusun dari kecil ke besar.

Class ini digunakan untuk mengimplementasikan algoritma Insertion Sort. Saat dijalankan, program akan menampilkan array sebelum diurutkan, kemudian melakukan proses sorting dengan membandingkan dan menyisipkan elemen ke posisi yang benar. Setelah proses selesai, program akan menampilkan array yang sudah terurut secara ascending.

Output:



```
<terminated> InsertionSort_2511531017 [Java Application]  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

Gambar 2.1

2.2 SelectionSort_2511531017

```
package pekan7_2511531017;

public class SelectionSort_2511531017 {
    public static void selectionSort_2511531017(int[] arr_1017) {
        int n_1017 = arr_1017.length;
        for(int i_1017 = 0; i_1017 < n_1017; i_1017++) {
            int minIndex_1017 = i_1017;
            for(int j_1017 = i_1017 + 1; j_1017 < n_1017; j_1017++) {
                if(arr_1017[j_1017] < arr_1017[minIndex_1017]) {
                    minIndex_1017 = j_1017;
                }
            }
            int temp_1017 = arr_1017[i_1017];
            arr_1017[i_1017] = arr_1017[minIndex_1017];
            arr_1017[minIndex_1017] = temp_1017;
        }
    }

    public static void main(String[] args) {
        int arr_1017[] = {23, 78, 45, 8, 32, 56, 1};
        int n_1017 = arr_1017.length;
        System.out.printf("array yang belum terurut:\n");
        for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
            System.out.print(arr_1017[i_1017] + " ");
        System.out.println("");
        selectionSort_2511531017(arr_1017);
        System.out.printf("array yang terurut:\n");
        for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
            System.out.print(arr_1017[i_1017] + " ");
        System.out.println("");
    }
}
```

Kode 2.2

Penjelasan:

```
public static void selectionSort_2511531017(int[] arr_1017) {
    int n_1017 = arr_1017.length;
    for(int i_1017 = 0; i_1017 < n_1017; i_1017++) {
        int minIndex_1017 = i_1017;
```

Memulai proses pengurutan data menggunakan algoritma Selection Sort. Variabel `n_1017` digunakan untuk menyimpan panjang array, sedangkan perulangan `for` digunakan untuk menentukan posisi elemen yang akan diisi dengan nilai terkecil. Variabel `minIndex_1017` digunakan untuk menyimpan indeks nilai terkecil sementara.

```
        for(int j_1017 = i_1017 + 1; j_1017 < n_1017; j_1017++) {
            if(arr_1017[j_1017] < arr_1017[minIndex_1017]) {
                minIndex_1017 = j_1017;
```

Mencari nilai terkecil pada bagian array yang belum terurut. Jika ditemukan elemen yang lebih kecil dari nilai pada `minIndex_1017`, maka indeks minimum akan diperbarui.

```
        int temp_1017 = arr_1017[i_1017];
        arr_1017[i_1017] = arr_1017[minIndex_1017];
        arr_1017[minIndex_1017] = temp_1017;
```

Untuk menukar posisi elemen saat ini dengan elemen terkecil yang ditemukan. Variabel temp_1017 digunakan sebagai penyimpanan sementara saat proses pertukaran data dilakukan.

```
public static void main(String[] args) {
    int arr_1017[] = {23, 78, 45, 8, 32, 56, 1};
    int n_1017 = arr_1017.length;
```

Digunakan untuk membuat method utama program dan mendeklarasikan array yang akan diurutkan. Variabel n_1017 digunakan untuk menyimpan jumlah elemen array.

```
System.out.printf("array yang belum terurut:\n");
for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
    System.out.print(arr_1017[i_1017] + " ");
System.out.println("");
```

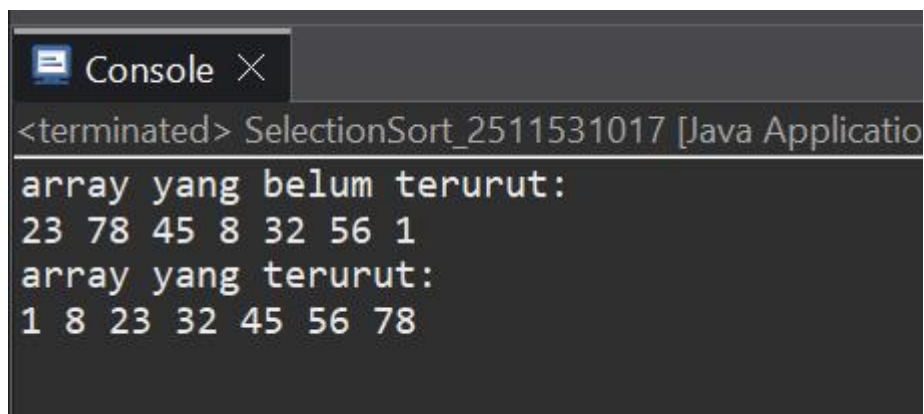
Menampilkan isi array sebelum dilakukan proses sorting. Program melakukan perulangan untuk mencetak seluruh elemen array ke layar.

```
System.out.printf("array yang terurut:\n");
for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
    System.out.print(arr_1017[i_1017] + " ");
System.out.println("");
```

Menampilkan isi array setelah proses pengurutan selesai dilakukan. Program akan mencetak seluruh elemen array yang sudah tersusun dari kecil ke besar.

Class ini digunakan untuk mengimplementasikan algoritma Selection Sort pada array integer. Saat dijalankan, program akan mencari nilai terkecil pada array lalu menukarnya ke posisi yang sesuai secara bertahap hingga seluruh data menjadi terurut. Setelah proses selesai, program akan menampilkan array sebelum dan sesudah diurutkan.

Output:



```
<terminated> SelectionSort_2511531017 [Java Applicatio
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Gambar 2.2

2.3 BubleSort_2511531017

```
package pekan7_2511531017;

public class BubleSort_2511531017 {
    public static void bubbleSort_2511531017(int[] arr_1017) {
        int n_1017 = arr_1017.length;
        for(int i_1017 = 0; i_1017 < n_1017; i_1017++) {
            for(int j_1017 = 0; j_1017 < n_1017 - i_1017 - 1; j_1017++)
            {
                if(arr_1017[j_1017] > arr_1017[j_1017 + 1]) {
```



```

        int temp_1017 = arr_1017[j_1017];
        arr_1017[j_1017] = arr_1017[j_1017 + 1];
        arr_1017[j_1017 + 1] = temp_1017;
    }
}
}
}
}
public static void main(String[] args) {
    int arr_1017[] = {23, 78, 45, 8, 32, 56, 1};
    int n_1017 = arr_1017.length;
    System.out.print("array yang belum terurut:\n");
    for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
        System.out.print(arr_1017[i_1017] + " ");
    System.out.println("");
    bubbleSort_2511531017(arr_1017);
    System.out.print("array yang terurut menggunakan BubleSort:\n");
    for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
        System.out.print(arr_1017[i_1017] + " ");
    System.out.println("");
}
}

```

Kode 2.3

Penjelasan:

```

public static void bubbleSort_2511531017(int[] arr_1017) {
    int n_1017 = arr_1017.length;
    for(int i_1017 = 0; i_1017 < n_1017; i_1017++) {

```

Memulai proses pengurutan data menggunakan algoritma Bubble Sort. Perulangan pertama digunakan untuk menentukan jumlah tahap sorting, sedangkan perulangan kedua digunakan untuk membandingkan elemen array yang berdekatan.

```

        for(int j_1017 = 0; j_1017 < n_1017 - i_1017 - 1; j_1017++) {
            if(arr_1017[j_1017] > arr_1017[j_1017 + 1]) {
                int temp_1017 = arr_1017[j_1017];
                arr_1017[j_1017] = arr_1017[j_1017 + 1];
                arr_1017[j_1017 + 1] = temp_1017;

```

Menukar posisi dua elemen jika elemen sebelah kiri lebih besar daripada elemen sebelah kanan. Variabel temp_1017 digunakan sebagai penyimpanan sementara saat proses pertukaran data dilakukan.

```

public static void main(String[] args) {
    int arr_1017[] = {23, 78, 45, 8, 32, 56, 1};
    int n_1017 = arr_1017.length;

```

Untuk membuat method utama program dan mendeklarasikan array yang akan diurutkan. Variabel n_1017 digunakan untuk menyimpan jumlah elemen array.

```

System.out.print("array yang belum terurut:\n");
for(int i_1017 = 0; i_1017 < n_1017; i_1017++)
    System.out.print(arr_1017[i_1017] + " ");
System.out.println("");

```

Menampilkan isi array sebelum dilakukan proses sorting. Program melakukan perulangan untuk mencetak seluruh elemen array ke layar.

```
System.out.print("array yang terurut menggunakan BubleSort:\n");
    for(int i_1017 =0; i_1017 < n_1017; i_1017++)
        System.out.print(arr_1017[i_1017] + " ");
    System.out.println("");
```

Menampilkan isi array setelah proses pengurutan selesai dilakukan. Program akan mencetak seluruh elemen array yang sudah tersusun dari kecil ke besar menggunakan Bubble Sort.

Class ini digunakan untuk mengimplementasikan algoritma Bubble Sort pada array integer. Saat dijalankan, program akan membandingkan dua elemen yang berdekatan lalu menukarnya jika urutannya salah hingga seluruh data menjadi terurut. Setelah proses selesai, program akan menampilkan array sebelum dan sesudah diurutkan.

Output:

```
<terminated> BubleSort_2511531017 [Java Application] C:\Users...
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut menggunakan BubleSort:
1 8 23 32 45 56 78
```

Gambar 2.3

2.4

InsertionSortGUI_2511531017

```
package pekan7_2511531017;

import java.awt.BorderLayout;

public class InsertionSortGUI_2511531017 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_1017;
    private JLabel[] labelArray_1017;
    JButton stepButton_1017;
    private JButton resetButton_1017;
    JButton setButton_1017;
    private JTextField inputField_1017;
    private JPanel panelArray_1017;
    private JTextArea stepArea_1017;

    private int i_1017 = 1, j_1017;
    private boolean sorting_1017 = false;
    private int stepCount_1017 = 1;

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511531017() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750,400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        JPanel inputPanel_1017 = new JPanel(new FlowLayout());
        inputField_1017 = new JTextField(30);
        setButton_1017 = new JButton("Set Array");
        inputPanel_1017.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
```

```

inputPanel_1017.add(inputField_1017);
inputPanel_1017.add(setButton_1017);

panelArray_1017 = new JPanel();
panelArray_1017.setLayout(new FlowLayout());

JPanel controlPanel_3023 = new JPanel();
stepButton_1017 = new JButton ("Langkah Selanjutnya ");
resetButton_1017 = new JButton ("Reset");
stepButton_1017.setEnabled(false);
controlPanel_3023.add(stepButton_1017);
controlPanel_3023.add(resetButton_1017);

stepArea_1017 = new JTextArea(8, 60);
stepArea_1017.setEditable(false);
stepArea_1017.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane = new JScrollPane(stepArea_1017);

add(inputPanel_1017, BorderLayout.NORTH);
add(panelArray_1017, BorderLayout.CENTER);
add(controlPanel_3023, BorderLayout.SOUTH);
add(scrollPane, BorderLayout.EAST);

setButton_1017.addActionListener(e -> setArrayFromInput_1017());
stepButton_1017.addActionListener(e -> performStep_1017());
resetButton_1017.addActionListener(e -> reset_1017());

}

private void setArrayFromInput_1017() {
    String text_1017 = inputField_1017.getText().trim();
    if(text_1017.isEmpty()) return;
    String[] parts_1017 = text_1017.split(",");
    array_1017 = new int [parts_1017.length];
    try {
        for (int k = 0; k < parts_1017.length; k++) {
            array_1017[k] = Integer.parseInt(parts_1017[k].trim());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukan hanya angka dipisahkan "+ "dengan koma!", "Error",
                JOptionPane.ERROR_MESSAGE);
            return; }
    }
    i_1017 = 1;
    stepCount_1017 = 1;
    sorting_1017 = true;
    stepButton_1017.setEnabled(true);
    stepArea_1017.setText("");
    panelArray_1017.removeAll();
    labelArray_1017 = new JLabel [array_1017.length];
    for (int k = 0; k < array_1017.length; k++) {
        labelArray_1017[k] = new JLabel(String.valueOf(array_1017[k]));
        labelArray_1017[k].setFont(new Font("Arial",Font.BOLD,24));
        labelArray_1017[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_1017[k].setPreferredSize(new Dimension (50,50));
        labelArray_1017[k].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_1017.add(labelArray_1017[k]);
    }
    panelArray_1017.revalidate();
    panelArray_1017.repaint();
}

private void performStep_1017() {
    if (i_1017 < array_1017.length && sorting_1017) {
        int key_1017 = array_1017[i_1017];
        j_1017 = i_1017 - 1;

```

```

        StringBuilder stepLog_1017 = new StringBuilder();
        stepLog_1017.append("Langkah").append(stepCount_1017).append(": Memasukkan ").append(key_1017).append("\n");

        while (j_1017 >= 0 && array_1017[j_1017] > key_1017) {
            array_1017[j_1017 + 1] = array_1017[j_1017];
            j_1017--;
        }
        array_1017[j_1017 + 1] = key_1017;

        updateLabels_1017();
        stepLog_1017.append("Hasil: ").append(arrayToString_1017(array_1017)).append("\n\n");

        i_1017++;
        stepCount_1017++;

        if (i_1017 == array_1017.length) {
            sorting_1017 = false;
            stepButton_1017.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
        }
    }

    private void updateLabels_1017() {
        for (int k = 0; k < array_1017.length; k++) {
            labelArray_1017[k].setText(String.valueOf(array_1017[k]));
        }
    }

    private void reset_1017() {
        inputField_1017.setText("");
        panelArray_1017.removeAll();
        panelArray_1017.revalidate();
        panelArray_1017.repaint();
        stepArea_1017.setText("");
        stepButton_1017.setEnabled(false);
        sorting_1017 = false;
        i_1017 = 1;
        stepCount_1017 = 1;
    }

    private String arrayToString_1017(int[] arr_1017) {
        StringBuilder sb = new StringBuilder();
        for (int k = 0; k < arr_1017.length; k++) {
            sb.append(arr_1017[k]);
            if (k < arr_1017.length - 1) sb.append(", ");
        }
        return sb.toString();
    }

    public static void main(String [] args) {
        SwingUtilities.invokeLater(() -> {
            InsertionSortGUI_2511531017 gui_1017 = new InsertionSortGUI_2511531017();
            gui_1017.setVisible(true);
        });
    }
}

```

Kode 2.4

Penjelasan:

```

public class InsertionSortGUI_2511531017 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_1017;
    private JLabel[] labelArray_1017;
    private JButton stepButton_1017;
    private JButton resetButton_1017;
    private JButton setButton_1017;
    private JTextField inputField_1017;
    private JPanel panelArray_1017;
    private JTextArea stepArea_1017;

```

Mendeklarasikan variabel dan komponen GUI yang dibutuhkan program.

```

    private int i_1017 = 1, j_1017;
    private boolean sorting_1017 = false;
    private int stepCount_1017 = 1;

```

Variabel `i_1017` dan `j_1017` digunakan sebagai indeks sorting, sedangkan `stepCount_1017` digunakan untuk menghitung jumlah langkah sorting.

```
public InsertionSortGUI_2511531017() {
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750,400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());
}
```

Konstruktor yang digunakan untuk mengatur tampilan utama GUI. Program mengatur judul window, ukuran frame, posisi tampilan, dan layout yang digunakan.

```
JPanel inputPanel_1017 = new JPanel(new FlowLayout());
    inputField_1017 = new JTextField(30);
    setButton_1017 = new JButton("Set Array");
    inputPanel_1017.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
```

Kode ini digunakan untuk membuat bagian input array pada GUI. Pengguna dapat memasukkan angka ke dalam text field dengan format dipisahkan tanda koma.

```
panelArray_1017 = new JPanel();
panelArray_1017.setLayout(new FlowLayout());
```

Membuat panel yang berfungsi menampilkan isi array pada GUI. Data array nantinya akan ditampilkan dalam bentuk label angka.

```
JPanel controlPanel_3023 = new JPanel();
    stepButton_1017 = new JButton("Langkah Selanjutnya ");
    resetButton_1017 = new JButton("Reset");
    stepButton_1017.setEnabled(false);
```

Digunakan untuk membuat tombol kontrol program. Tombol langkah digunakan untuk menjalankan sorting satu tahap, sedangkan tombol reset digunakan untuk mengembalikan program ke kondisi awal.

```
stepArea_1017 = new JTextArea(8, 60);
stepArea_1017.setEditable(false);
```

Membuat area teks yang menampilkan langkah-langkah sorting. Area teks dibuat tidak bisa diedit oleh pengguna.

```
stepArea_1017.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane = new JScrollPane(stepArea_1017);
```

Mengatur font pada area langkah sorting dan menambahkan scroll pane agar teks dapat discroll.

```
add(inputPanel_1017, BorderLayout.NORTH);
add(panelArray_1017, BorderLayout.CENTER);
add(controlPanel_3023, BorderLayout.SOUTH);
add(scrollPane, BorderLayout.EAST);
```

Menempatkan semua komponen GUI pada posisi layout masing-masing.

```
setButton_1017.addActionListener(e -> setArrayFromInput_1017());
stepButton_1017.addActionListener(e -> performStep_1017());
resetButton_1017.addActionListener(e -> reset_1017());
```

Memberikan aksi pada setiap tombol GUI.

```
private void setArrayFromInput_1017() {
    String text_1017 = inputField_1017.getText().trim();
}
```

```

if(text_1017.isEmpty()) return;
String[] parts_1017 = text_1017.split(",");
array_1017 = new int [parts_1017.length];

```

Untuk mengambil input angka dari pengguna lalu memisahkannya berdasarkan tanda koma. Setelah itu program membuat array integer sesuai jumlah data yang dimasukkan.

```

try {
    for (int k = 0; k < parts_1017.length; k++) {
        array_1017[k] =
Integer.parseInt(parts_1017[k].trim());}
    }catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukan hanya
angka dipisahkan "+ "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
        return; }

```

mengubah input string menjadi integer dan menangani kesalahan input. Jika pengguna memasukkan selain angka, program akan menampilkan pesan error menggunakan JOptionPane.

```

i_1017 = 1;
stepCount_1017 = 1;
sorting_1017 = true;
stepButton_1017.setEnabled(true);
stepArea_1017.setText("");
panelArray_1017.removeAll();

```

Mengatur ulang kondisi awal proses sorting. Program juga membersihkan tampilan sebelumnya dan mengaktifkan tombol langkah.

```

labelArray_1017 = new JLabel [array_1017.length];
    for (int k = 0; k < array_1017.length; k++) {
        labelArray_1017[k] = new
JLabel(String.valueOf(array_1017[k]));
        labelArray_1017[k].setFont(new Font("Arial",Font.BOLD,24));
        labelArray_1017[k].setBorder(BorderFactory.createLineBorder(Color.BLACK
));
        labelArray_1017[k].setPreferredSize(new Dimension (50,50));
        labelArray_1017[k].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_1017.add(labelArray_1017[k]);

```

Digunakan untuk membuat visualisasi array pada GUI menggunakan label. Setiap elemen array ditampilkan dalam bentuk kotak angka dengan ukuran, border, dan posisi teks yang sudah diatur.

```

panelArray_1017.revalidate();
panelArray_1017.repaint();

```

Memperbarui tampilan panel setelah array berhasil ditampilkan ke GUI.

```

private void performStep_1017() {
    if (i_1017 < array_1017.length && sorting_1017) {
        int key_1017 = array_1017[i_1017];
        j_1017 = i_1017 - 1;

```


Untuk menjalankan satu langkah proses Insertion Sort. Program mengambil nilai key_1017 lalu membandingkannya dengan elemen sebelumnya.

```
StringBuilder stepLog_1017 = new StringBuilder();
    stepLog_1017.append("Langkah").append(stepCount_1017).append(":
Memasukkan ").append(key_1017).append("\n")
```

Membuat dan menyimpan teks langkah sorting yang nantinya ditampilkan pada area teks GUI.

```
while (j_1017 >= 0 && array_1017[j_1017] > key_1017) {
    array_1017[j_1017 + 1] = array_1017[j_1017];
    j_1017--;
}
array_1017[j_1017 + 1] = key_1017;
```

Untuk menggeser elemen yang lebih besar ke kanan lalu menempatkan key_1017 pada posisi yang benar. Bagian ini merupakan inti dari algoritma Insertion Sort.

```
updateLabels_1017();
    stepLog_1017.append("Hasil:
").append(arrayToString_1017(array_1017)).append("\n\n");
    i_1017++;
    stepCount_1017++;
```

Memperbarui tampilan array pada GUI serta menampilkan hasil langkah sorting ke area teks. Setelah itu indeks dan jumlah langkah sorting akan ditambah.

```
if (i_1017 == array_1017.length) {
    sorting_1017 = false;
    stepButton_1017.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Sorting
selesai!");
```

Untuk mengecek apakah proses sorting sudah selesai. Jika selesai, tombol langkah akan dinonaktifkan dan program menampilkan pesan bahwa sorting telah selesai.

```
private void updateLabels_1017() {
    for (int k = 0; k < array_1017.length; k++) {
        labelArray_1017[k].setText(String.valueOf(array_1017[k]));
```

Memperbarui isi label array pada GUI agar sesuai dengan hasil sorting terbaru.

```
private void reset_1017() {
    inputField_1017.setText("");
    panelArray_1017.removeAll();
    panelArray_1017.revalidate();
    panelArray_1017.repaint();
    stepArea_1017.setText("");
    stepButton_1017.setEnabled(false);
    sorting_1017 = false;
    i_1017 = 1;
    stepCount_1017 = 1;
```

Digunakan untuk mengembalikan program ke kondisi awal. Input, tampilan array, dan langkah sorting akan dibersihkan kembali.

```
private String arrayToString_1017(int[] arr_1017) {
    StringBuilder sb = new StringBuilder();
```

```

        for (int k = 0; k < arr_1017.length; k++) {
            sb.append(arr_1017[k]);
            if (k < arr_1017.length - 1) sb.append(", ");
        }
        return sb.toString();
    }

```

Mengubah isi array menjadi string agar dapat ditampilkan pada area langkah sorting.

```

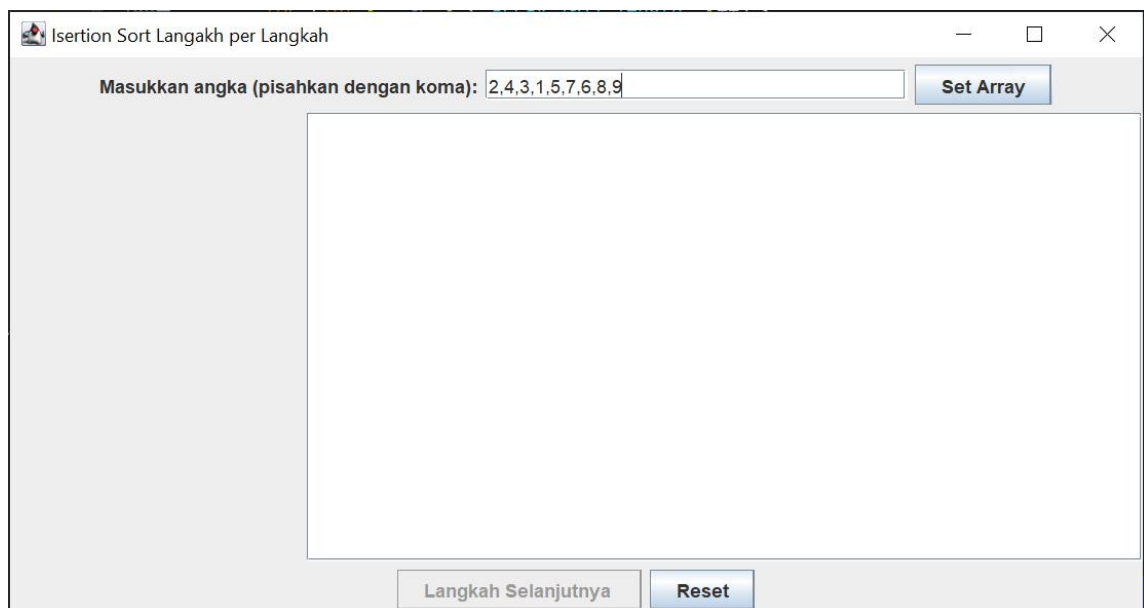
public static void main(String [] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511531017 gui_1017 = new
InsertionSortGUI_2511531017();
        gui_1017.setVisible(true);
    });
}

```

Digunakan untuk menjalankan program GUI Insertion Sort dan menampilkan window aplikasi ke layar.

Class ini digunakan untuk memvisualisasikan algoritma Insertion Sort menggunakan GUI Java Swing. Pengguna dapat memasukkan data array lalu menjalankan proses sorting langkah demi langkah menggunakan tombol yang tersedia sehingga proses pengurutan lebih mudah dipahami secara visual. Program akan menampilkan tampilan GUI berupa input array, tombol kontrol, area langkah sorting, dan visualisasi array. Setelah pengguna memasukkan angka dan menekan tombol langkah selanjutnya, program akan menampilkan proses pengurutan data secara bertahap hingga seluruh array tersusun dari kecil ke besar.

Output:



Isertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

2	4	3
1	5	7
6	8	9

Isertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

2	3	4
1	5	7
6	8	9

Isertion Sort Langkah per Langkah

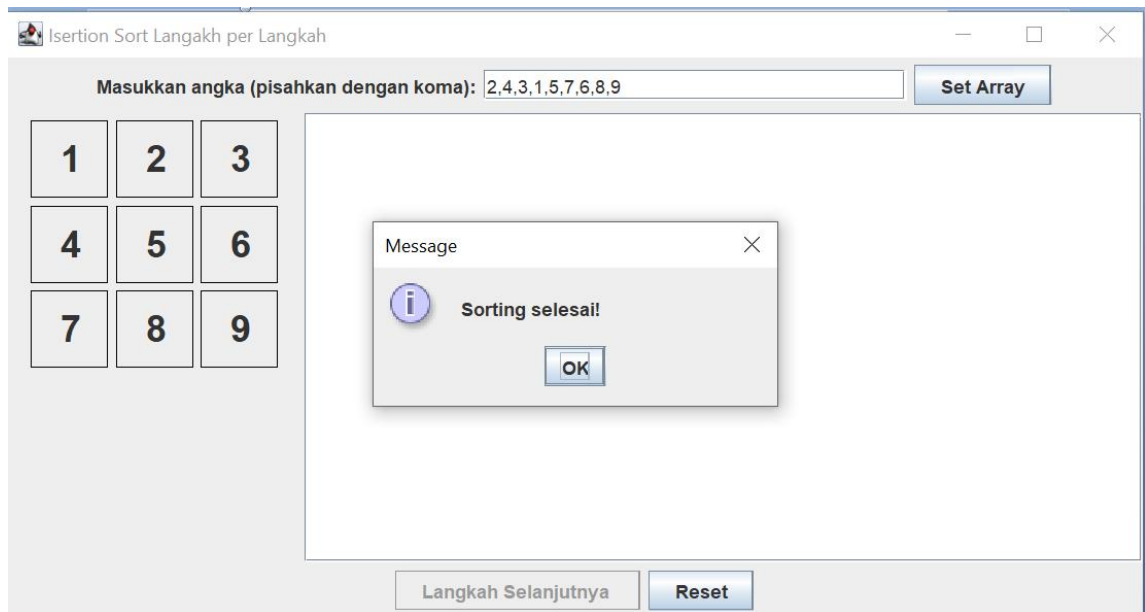
Masukkan angka (pisahkan dengan koma):

1	2	3
4	5	7
6	8	9

Isertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

1	2	3
4	5	6
7	8	9



Gambar 2.4

TUGAS

2.5 Mahasiswa_2511531017

```
package pekan7_2511531017;

public class Mahasiswa_2511531017 {
    private String nama_1017;
    private String nim_1017;
    private String prodi_1017;

    public Mahasiswa_2511531017(String nama_1017, String nim_1017, String
prodi_1017) {
        this.nama_1017 = nama_1017;
        this.nim_1017 = nim_1017;
        this.prodi_1017 = prodi_1017;
    }
    public String getNama_1017() {
        return nama_1017;
    }
    public void setNama_1017(String nama_1017) {
        this.nama_1017 = nama_1017;
    }
    public String getNim_1017() {
        return nim_1017;
    }
    public void setNim_1017(String nim_1017) {
        this.nim_1017 = nim_1017;
    }
    public String getProdi_1017() {
        return prodi_1017;
    }
    public void setProdi_1017(String prodi_1017) {
        this.prodi_1017 = prodi_1017;
    }
    public String toString() {
        return nama_1017 + " | " + nim_1017 + " | " + prodi_1017;
    }
}
```

Kode 2.5

Penjelasan:

```
public Mahasiswa_2511531017(String nama_1017, String nim_1017, String
prodi_1017) {
    this.nama_1017 = nama_1017;
    this.nim_1017 = nim_1017;
    this.prodi_1017 = prodi_1017;
}
```

Konstruktor yang digunakan untuk mengisi nilai atribut saat object mahasiswa dibuat. Keyword this digunakan untuk membedakan variabel parameter dengan atribut class.

```
public String getNama_1017() {
    return nama_1017;
}
public void setNama_1017(String nama_1017) {
    this.nama_1017 = nama_1017;
}
```

Method getter dan setter untuk mengambil dan mengubah nilai atribut nama_1017.

```
public String getNim_1017() {  
    return nim_1017;  
}  
public void setNim_1017(String nim_1017) {  
    this.nim_1017 = nim_1017;  
}
```

Method getter dan setter untuk mengambil dan mengubah nilai atribut nim_1017.

```
public String getProdi_1017() {  
    return prodi_1017;  
}  
public void setProdi_1017(String prodi_1017) {  
    this.prodi_1017 = prodi_1017;  
}
```

Method getter dan setter untuk mengambil dan mengubah nilai atribut prodi_1017

```
public String toString() {  
    return nama_1017 + " | " + nim_1017 + " | " + prodi_1017;  
}
```

Menampilkan data mahasiswa dalam bentuk string. Method toString() akan dipanggil otomatis saat object mahasiswa ditampilkan.

Class ini digunakan untuk membuat ADT Mahasiswa yang menyimpan data nama, NIM, dan program studi mahasiswa. Class ini nantinya digunakan pada program sorting GUI sebagai object yang akan diurutkan berdasarkan nama mahasiswa.

2.6 MahasiswaGUI_2511531017

```
package pekan7_2511531017;  
  
import java.awt.BorderLayout;  
import java.awt.FlowLayout;  
import java.util.ArrayList;  
  
import javax.swing.JButton;  
import javax.swing.JComboBox;  
import javax.swing.JFrame;  
import javax.swing.JLabel;  
import javax.swing.JOptionPane;  
import javax.swing.JPanel;  
import javax.swing.JScrollPane;  
import javax.swing.JTable;  
import javax.swing.JTextArea;  
import javax.swing.JTextField;  
import javax.swing.SwingUtilities;  
import javax.swing.table.DefaultTableModel;  
  
public class MahasiswaGUI_2511531017 extends JFrame {  
  
    private ArrayList<Mahasiswa_2511531017> data_1017 = new ArrayList<>();  
    private JTextField namaField_1017;  
    private JTextField nimField_1017;  
    private JTextField prodiField_1017;  
    private JButton tambahButton_1017;  
    private JButton hapusButton_1017;  
    private JButton sortingButton_1017;  

```

```

private JButton langkahButton_1017;
private JButton resetButton_1017;
private JComboBox<String> comboSorting_1017;
private JTable table_1017;
private DefaultTableModel model_1017;
private JTextArea prosesArea_1017;

private int i_1017 = 0;
private int j_1017 = 0;
private boolean sorting_1017 = false;

public MahasiswaGUI_2511531017() {
    setTitle("Sorting Mahasiswa 2511531017");
    setSize(950, 500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    JPanel inputPanel_1017 = new JPanel(new FlowLayout());
    namaField_1017 = new JTextField(10);
    nimField_1017 = new JTextField(10);
    prodiField_1017 = new JTextField(10);

    tambahButton_1017 = new JButton("Tambah");
    hapusButton_1017 = new JButton("Hapus");
    sortingButton_1017 = new JButton("Mulai Sorting");
    langkahButton_1017 = new JButton("Langkah Selanjutnya");
    resetButton_1017 = new JButton("Reset");

    langkahButton_1017.setEnabled(false);
    comboSorting_1017 = new JComboBox<>();
    comboSorting_1017.addItem("Insertion Sort");
    comboSorting_1017.addItem("Selection Sort");
    comboSorting_1017.addItem("Bubble Sort");

    inputPanel_1017.add(new JLabel("Nama"));
    inputPanel_1017.add(namaField_1017);
    inputPanel_1017.add(new JLabel("NIM"));
    inputPanel_1017.add(nimField_1017);
    inputPanel_1017.add(new JLabel("Prodi"));
    inputPanel_1017.add(prodiField_1017);
    inputPanel_1017.add(tambahButton_1017);
    inputPanel_1017.add(hapusButton_1017);
    inputPanel_1017.add(comboSorting_1017);
    inputPanel_1017.add(sortingButton_1017);
    inputPanel_1017.add(langkahButton_1017);
    inputPanel_1017.add(resetButton_1017);

    add(inputPanel_1017, BorderLayout.NORTH);

    model_1017 = new DefaultTableModel();
    model_1017.addColumn("Nama");
    model_1017.addColumn("NIM");
    model_1017.addColumn("Prodi");
    table_1017 = new JTable(model_1017);

    JScrollPane tableScroll_1017 = new JScrollPane(table_1017);

```

```

        add(tableScroll_1017, BorderLayout.CENTER);
        prosesArea_1017 = new JTextArea();
        prosesArea_1017.setEditable(false);
        JScrollPane prosesScroll_1017 = new JScrollPane(prosesArea_1017);
        add(prosesScroll_1017, BorderLayout.EAST);

        tambahButton_1017.addActionListener(e -> tambahData_1017());
        hapusButton_1017.addActionListener(e -> hapusData_1017());
        sortingButton_1017.addActionListener(e -> mulaiSorting_1017());
        langkahButton_1017.addActionListener(e -> langkahSorting_1017());
        resetButton_1017.addActionListener(e -> reset_1017());
    }

    private void tambahData_1017() {
        String nama_1017 = namaField_1017.getText();
        String nim_1017 = nimField_1017.getText();
        String prodi_1017 = prodiField_1017.getText();
        if (nama_1017.isEmpty() || nim_1017.isEmpty() || prodi_1017.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Input tidak boleh kosong!");
            return;
        }

        Mahasiswa_2511531017 mhs_1017 = new
Mahasiswa_2511531017(nama_1017, nim_1017, prodi_1017);
        data_1017.add(mhs_1017);
        model_1017.addRow(new Object[]{nama_1017, nim_1017, prodi_1017
        });

        namaField_1017.setText("");
        nimField_1017.setText("");
        prodiField_1017.setText("");
    }

    private void hapusData_1017() {
        int row_1017 = table_1017.getSelectedRow();
        if (row_1017 >= 0) {
            data_1017.remove(row_1017);
            model_1017.removeRow(row_1017);
        } else {
            JOptionPane.showMessageDialog(this, "Pilih data yang ingin
dihapus!");
        }
    }

    private void mulaiSorting_1017() {
        i_1017 = 0;
        sorting_1017 = true;
        prosesArea_1017.setText("");
        langkahButton_1017.setEnabled(true);
        JOptionPane.showMessageDialog(this, "Klik tombol Langkah Selanjutnya");
    }

    private void langkahSorting_1017() {
        String pilihan_1017 = comboSorting_1017.getSelectedItem().toString();
        if (pilihan_1017.equals("Insertion Sort")) {
            langkahInsertion_1017();
        } else if (pilihan_1017.equals("Selection Sort")) {

```

```

        langkahSelection_1017();
    } else {
        langkahBubble_1017();
    }
    refreshTable_1017();
}

private void langkahInsertion_1017() {
    prosesArea_1017.append( "=== INSERTION SORT ===\n");
    if (i_1017 < data_1017.size()) {
        Mahasiswa_2511531017 key_1017 = data_1017.get(i_1017);
        j_1017 = i_1017 - 1;
        while (j_1017 >= 0 &&
data_1017.get(j_1017).getNama_1017().compareToIgnoreCase(key_1017.getNama_101
7()) > 0) {
            data_1017.set(j_1017 + 1,data_1017.get(j_1017));
            j_1017--;
        }
        data_1017.set(j_1017 + 1,key_1017);
        prosesArea_1017.append("Langkah "+ i_1017+ " : "+ data_1017+
"\n");
        i_1017++;
    } else {
        selesai_1017();
    }
}

private void langkahSelection_1017() {
    prosesArea_1017.append("=== SELECTION SORT ===\n");
    if (i_1017 < data_1017.size()) {
        int minIndex_1017 = i_1017;
        for (int j_1017 = i_1017 + 1;j_1017 < data_1017.size();j_1017++)
{
            if
(data_1017.get(j_1017).getNama_1017().compareToIgnoreCase(data_1017.get(minIn
dex_1017).getNama_1017()) < 0) {
                minIndex_1017 = j_1017;
            }
        }
        Mahasiswa_2511531017 temp_1017 = data_1017.get(i_1017);
        data_1017.set(i_1017,data_1017.get(minIndex_1017));
        data_1017.set(minIndex_1017,temp_1017);
        prosesArea_1017.append( "Pass "+ (i_1017 + 1)+ " : "+ data_1017+
"\n");
        i_1017++;
    } else {
        selesai_1017();
    }
}

private void langkahBubble_1017() {
    prosesArea_1017.append( "=== BUBBLE SORT ===\n");
    if (i_1017 < data_1017.size()) {
        for (int j_1017 = 0;j_1017 < data_1017.size()- i_1017 -
1;j_1017++) {
            if
(data_1017.get(j_1017).getNama_1017().compareToIgnoreCase( data_1017.get(j_10
17 + 1).getNama_1017()) > 0) {

```



```

        Mahasiswa_2511531017 temp_1017 = data_1017.get(j_1017);
        data_1017.set(j_1017, data_1017.get(j_1017 + 1));
        data_1017.set(j_1017 + 1, temp_1017);
    }
}
prosesArea_1017.append("Pass " + (i_1017 + 1) + " : " + data_1017+
"\n");
    i_1017++;
} else {
    selesai_1017();
}
}

private void selesai_1017() {
    sorting_1017 = false;
    langkahButton_1017.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Sorting selesai!");
}

private void refreshTable_1017() {
    model_1017.setRowCount(0);
    for (Mahasiswa_2511531017 mhs_1017: data_1017) {
        model_1017.addRow(new Object[]{
            mhs_1017.getNama_1017(),
            mhs_1017.getNim_1017(),
            mhs_1017.getProdi_1017()
        });
    }
}

private void reset_1017() {
    data_1017.clear();
    model_1017.setRowCount(0);
    prosesArea_1017.setText("");
    namaField_1017.setText("");
    nimField_1017.setText("");
    prodiField_1017.setText("");
    langkahButton_1017.setEnabled(false);
    i_1017 = 0;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MahasiswaGUI_2511531017 gui_1017 = new MahasiswaGUI_2511531017();
        gui_1017.setVisible(true);
    });
}
}

```

Kode 2.6

Penjelasan:

```

public MahasiswaGUI_2511531017() {
    setTitle("Sorting Mahasiswa 2511531017");
    setSize(950, 500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
}

```

```
setLayout(new BorderLayout());
```

Konstruktor yang digunakan untuk mengatur tampilan utama GUI. Program mengatur ukuran window, posisi tampilan, dan layout aplikasi. Constructor akan dijalankan otomatis saat object GUI dibuat.

```
JPanel inputPanel_1017 = new JPanel(new FlowLayout());  
namaField_1017 = new JTextField(10);  
nimField_1017 = new JTextField(10);  
prodiField_1017 = new JTextField(10);
```

Membuat panel input dan text field mahasiswa. Panel input berfungsi sebagai tempat komponen input ditampilkan pada GUI. Sedangkan JTextField digunakan pengguna untuk memasukkan data mahasiswa.

```
tambahButton_1017 = new JButton("Tambah");  
hapusButton_1017 = new JButton("Hapus");  
sortingButton_1017 = new JButton("Mulai Sorting");  
langkahButton_1017 = new JButton("Langkah Selanjutnya");  
resetButton_1017 = new JButton("Reset");
```

Untuk membuat tombol-tombol pada GUI program. Setiap tombol memiliki fungsi tertentu sesuai kebutuhan pengguna. Tombol tersebut akan dihubungkan dengan method menggunakan action listener.

```
langkahButton_1017.setEnabled(false);  
comboSorting_1017 = new JComboBox<>();  
comboSorting_1017.addItem("Insertion Sort");  
comboSorting_1017.addItem("Selection Sort");  
comboSorting_1017.addItem("Bubble Sort");
```

Untuk menonaktifkan tombol langkah saat program pertama dijalankan. Selain itu, program juga membuat pilihan algoritma sorting menggunakan JComboBox. Pengguna dapat memilih algoritma Insertion Sort, Selection Sort, atau Bubble Sort.

```
inputPanel_1017.add(new JLabel("Nama"));  
inputPanel_1017.add(namaField_1017);  
inputPanel_1017.add(new JLabel("NIM"));  
inputPanel_1017.add(nimField_1017);  
inputPanel_1017.add(new JLabel("Prodi"));  
inputPanel_1017.add(prodiField_1017);
```

Menambahkan label dan field input ke panel GUI. Label digunakan sebagai penjelasan input yang harus diisi pengguna. Sedangkan text field digunakan sebagai tempat memasukkan data mahasiswa.

```
inputPanel_1017.add(tambahButton_1017);  
inputPanel_1017.add(hapusButton_1017);  
inputPanel_1017.add(comboSorting_1017);  
inputPanel_1017.add(sortingButton_1017);  
inputPanel_1017.add(langkahButton_1017);  
inputPanel_1017.add(resetButton_1017);
```

Menambahkan tombol dan combo box ke panel input GUI. Semua komponen kontrol program akan tampil pada bagian atas window. Pengguna dapat langsung menggunakan tombol tersebut untuk menjalankan program.

```
add(inputPanel_1017, BorderLayout.NORTH);
```

menempatkan panel input pada bagian atas GUI

```
model_1017 = new DefaultTableModel();
model_1017.addColumn("Nama");
model_1017.addColumn("NIM");
model_1017.addColumn("Prodi")
```

Membuat model tabel mahasiswa. Program menambahkan tiga kolom yaitu nama, NIM, dan program studi. Data mahasiswa nantinya akan ditampilkan pada tabel tersebut.

```
table_1017 = new JTable(model_1017);
JScrollPane tableScroll_1017 = new JScrollPane(table_1017);
add(tableScroll_1017, BorderLayout.CENTER);
```

Membuat tabel mahasiswa pada GUI. JScrollPane digunakan agar tabel dapat discroll jika data terlalu banyak. Tabel kemudian ditempatkan pada bagian tengah window.

```
prosesArea_1017 = new JTextArea();
prosesArea_1017.setEditable(false);
JScrollPane prosesScroll_1017 = new JScrollPane(prosesArea_1017);
add(prosesScroll_1017, BorderLayout.EAST);
```

Membuat area teks proses sorting. Area teks dibuat tidak dapat diedit agar pengguna hanya dapat melihat hasil proses. Komponen tersebut ditempatkan pada bagian kanan GUI.

```
tambahButton_1017.addActionListener(e -> tambahData_1017());
hapusButton_1017.addActionListener(e -> hapusData_1017());
sortingButton_1017.addActionListener(e -> mulaiSorting_1017());
langkahButton_1017.addActionListener(e -> langkahSorting_1017());
resetButton_1017.addActionListener(e -> reset_1017());
```

Untuk memberikan aksi pada setiap tombol GUI. Ketika tombol ditekan, method tertentu akan otomatis dijalankan. Dengan action listener program menjadi interaktif terhadap input pengguna.

```
private void tambahData_1017() {
    String nama_1017 = namaField_1017.getText();
    String nim_1017 = nimField_1017.getText();
    String prodi_1017 = prodiField_1017.getText();
```

Membuat method penambahan data mahasiswa. Method akan mengambil data dari text field lalu menyimpannya ke ArrayList dan tabel. Method ini dipanggil saat tombol tambah ditekan.

```
if (nama_1017.isEmpty() || nim_1017.isEmpty() || prodi_1017.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Input tidak boleh kosong!");
```

Memeriksa apakah ada input yang kosong. Jika ada field kosong maka program akan menampilkan pesan error menggunakan JOptionPane. Hal ini bertujuan agar data yang dimasukkan lengkap.

```
Mahasiswa_2511531017 mhs_1017 = new
Mahasiswa_2511531017(nama_1017, nim_1017, prodi_1017);
data_1017.add(mhs_1017);
```

Membuat object mahasiswa baru. Setelah object dibuat, data mahasiswa akan disimpan ke dalam ArrayList. Data tersebut nantinya akan diproses pada algoritma sorting.

```
model_1017.addRow(new Object[]{nama_1017,nim_1017,prodi_1017
});
```

Menambahkan data mahasiswa ke tabel GUI. Data yang dimasukkan pengguna akan langsung muncul pada tabel. Dengan demikian pengguna dapat melihat data yang sudah tersimpan.

```
private void hapusData_1017() {
    int row_1017 = table_1017.getSelectedRow();
    if (row_1017 >= 0) {
        data_1017.remove(row_1017);
        model_1017.removeRow(row_1017);
    } else {
        JOptionPane.showMessageDialog(this,"Pilih data yang ingin
dihapus!");
    }
}
```

Membuat method penghapusan data mahasiswa. Data yang dipilih pada tabel akan dihapus dari tabel dan ArrayList. Method ini dijalankan saat tombol hapus ditekan.

```
private void mulaiSorting_1017() {
    i_1017 = 0;
    sorting_1017 = true;
}
```

Memulai proses sorting data mahasiswa. Variabel indeks akan dikembalikan ke awal sebelum sorting dimulai. Program juga mengubah status sorting menjadi aktif.

```
langkahButton_1017.setEnabled(true);
JOptionPane.showMessageDialog(this,"Klik tombol Langkah Selanjutnya");
```

Untuk mengaktifkan tombol langkah sorting. Program juga menampilkan pesan instruksi kepada pengguna. Setelah itu pengguna dapat menjalankan sorting langkah demi langkah.

```
private void langkahSorting_1017() {
    String pilihan_1017 = comboSorting_1017.getSelectedItem().toString();
}
```

Menjalankan proses sorting sesuai algoritma yang dipilih pengguna. Method ini akan memanggil method sorting tertentu seperti insertion, selection, atau bubble sort. Method dijalankan saat tombol langkah ditekan.

```
if (pilihan_1017.equals("Insertion Sort")) {
    langkahInsertion_1017();
} else if (pilihan_1017.equals("Selection Sort")) {
    langkahSelection_1017();
} else {
    langkahBubble_1017();
}
```

Menentukan algoritma sorting yang dipilih pengguna. Program akan menjalankan method sorting sesuai pilihan pada combo box. Dengan demikian pengguna dapat mencoba beberapa algoritma sorting berbeda.

```
private void langkahInsertion_1017() {
    prosesArea_1017.append( "=== INSERTION SORT ===\n");
    if (i_1017 < data_1017.size()) {
        Mahasiswa_2511531017 key_1017 = data_1017.get(i_1017);
        j_1017 = i_1017 - 1;
    }
}
```

Menjalankan proses Insertion Sort secara bertahap. Program akan mengurutkan data mahasiswa berdasarkan nama. Hasil setiap langkah sorting akan ditampilkan pada area proses.

```
private void langkahSelection_1017() {
    prosesArea_1017.append("=== SELECTION SORT ===\n");
    if (i_1017 < data_1017.size()) {
        int minIndex_1017 = i_1017;
        for (int j_1017 = i_1017 + 1; j_1017 < data_1017.size(); j_1017++)
        {
            if
            (data_1017.get(j_1017).getNama_1017().compareToIgnoreCase(data_1017.get(minIndex_1017).getNama_1017()) < 0) {
                minIndex_1017 = j_1017;
            }
        }
        Mahasiswa_2511531017 temp_1017 = data_1017.get(i_1017);
        data_1017.set(i_1017, data_1017.get(minIndex_1017));
        data_1017.set(minIndex_1017, temp_1017);
        prosesArea_1017.append( "Pass "+ (i_1017 + 1)+ " : "+ data_1017+
        "\n");
        i_1017++;
    } else {
        selesai_1017();
    }
}
```

Kode ini digunakan untuk menjalankan Selection Sort langkah demi langkah. Program akan mencari data dengan nilai terkecil pada setiap proses. Setelah ditemukan maka data akan ditukar posisinya.

```
private void langkahBubble_1017() {
    prosesArea_1017.append( "=== BUBBLE SORT ===\n");
    if (i_1017 < data_1017.size()) {
        for (int j_1017 = 0; j_1017 < data_1017.size() - i_1017 - 1; j_1017++) {
            if
            (data_1017.get(j_1017).getNama_1017().compareToIgnoreCase( data_1017.get(j_1017 + 1).getNama_1017()) > 0) {
                Mahasiswa_2511531017 temp_1017 = data_1017.get(j_1017);
                data_1017.set(j_1017, data_1017.get(j_1017 + 1));
                data_1017.set(j_1017 + 1, temp_1017);
            }
        }
        prosesArea_1017.append("Pass "+ (i_1017 + 1)+ " : "+ data_1017+
        "\n");
        i_1017++;
    } else {
        selesai_1017();
    }
}
```

Menjalankan Bubble Sort secara bertahap. Program akan membandingkan dua data yang berdekatan. Jika urutannya salah maka kedua data akan ditukar. Membandingkan dua nama mahasiswa yang berdekatan pada Bubble Sort. Jika data kiri lebih besar maka program akan melakukan pertukaran data. Proses ini dilakukan berulang hingga data terurut.

```
private void selesai_1017() {
    sorting_1017 = false;
}
```

```

        langkahButton_1017.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }

```

Untuk menghentikan proses sorting ketika seluruh data sudah terurut. Method ini dipanggil setelah semua langkah sorting selesai dijalankan. Program kemudian menampilkan pesan bahwa sorting telah selesai. Juga menampilkan pesan informasi kepada pengguna. Dengan demikian pengguna mengetahui bahwa proses sorting telah selesai.

```

private void refreshTable_1017() {
    model_1017.setRowCount(0);
    for (Mahasiswa_2511531017 mhs_1017: data_1017) {
        model_1017.addRow(new Object[]{
            mhs_1017.getNama_1017(),
            mhs_1017.getNim_1017(),
            mhs_1017.getProdi_1017()
        });
    }
}

```

Memperbarui isi tabel setelah proses sorting dilakukan. setRowCount(0) digunakan untuk menghapus seluruh isi tabel sementara sebelum data baru dimasukkan kembali. Setelah itu, program melakukan perulangan untuk menampilkan seluruh data mahasiswa terbaru ke dalam tabel.

```

private void reset_1017() {
    data_1017.clear();
    model_1017.setRowCount(0);
    prosesArea_1017.setText("");
    namaField_1017.setText("");
    nimField_1017.setText("");
    prodiField_1017.setText("");
    langkahButton_1017.setEnabled(false);
    i_1017 = 0;
}

```

Mengembalikan program ke kondisi awal seperti saat pertama dijalankan. Seluruh data mahasiswa, isi tabel, proses sorting, dan input field akan dibersihkan kembali. Selain itu tombol langkah akan dinonaktifkan dan indeks sorting dikembalikan ke nilai awal.

```

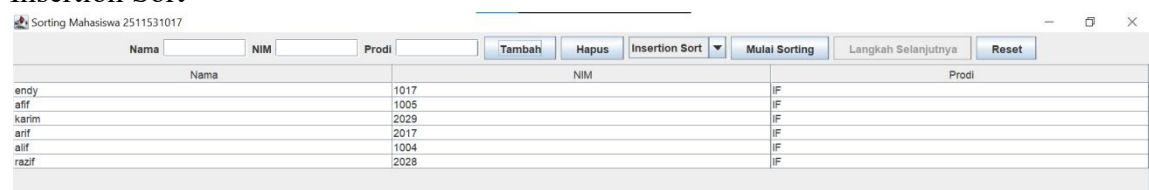
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MahasiswaGUI_2511531017 gui_1017 = new MahasiswaGUI_2511531017();
        gui_1017.setVisible(true);
    });
}

```

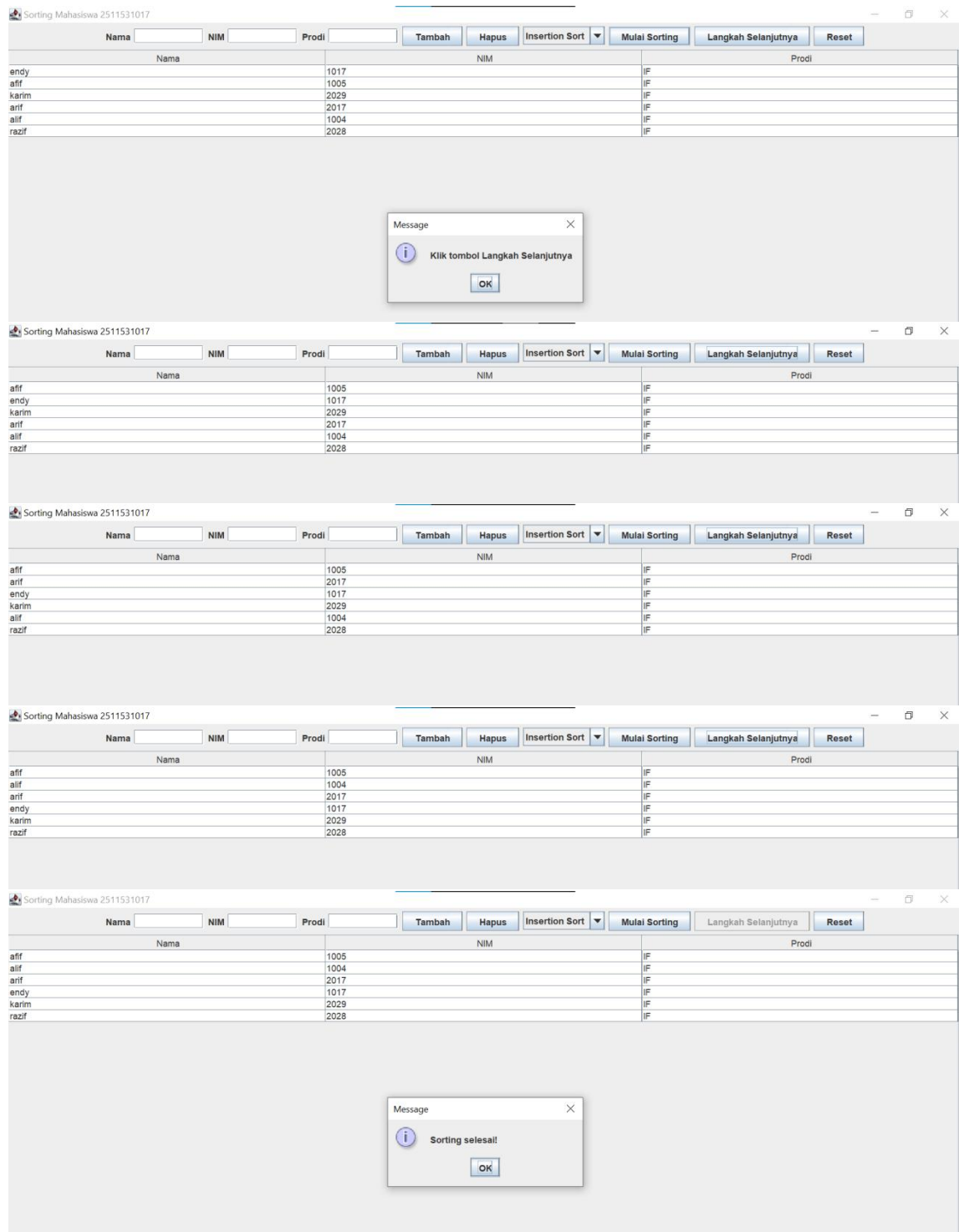
Method utama yang digunakan untuk menjalankan program GUI. SwingUtilities.invokeLater() digunakan agar GUI dijalankan pada thread Swing dengan aman. Setelah itu object MahasiswaGUI_2511531017 dibuat dan ditampilkan ke layar menggunakan setVisible(true).

Output:

Insertion Sort



Nama	NIM	Prodi
endy	1017	IF
arif	1005	IF
karim	2029	IF
arif	2017	IF
alif	1004	IF
razif	2028	IF



Selection Sort

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
a	1	if
c	2	if
d	3	if
f	4	if
b	5	if
e	6	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
a	1	if
b	5	if
d	3	if
f	4	if
c	2	if
e	6	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
a	1	if
b	5	if
c	2	if
f	4	if
d	3	if
e	6	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
a	1	if
b	5	if
c	2	if
d	3	if
f	4	if
e	6	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
a	1	if
b	5	if
c	2	if
d	3	if
e	6	if
f	4	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
a	1	if
b	5	if
c	2	if
d	3	if
e	6	if
f	4	if

Message

Sorting selesai!

Buble Sort

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
budi	1	if
asep	2	if
sarah	3	if
yanto	4	if
ratna	5	if
amaik	6	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
asep	2	if
budi	1	if
sarah	3	if
ratna	5	if
amaik	6	if
yanto	4	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
asep	2	if
budi	1	if
ratna	5	if
amaik	6	if
sarah	3	if
yanto	4	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
asep	2	if
budi	1	if
amaik	6	if
ratna	5	if
sarah	3	if
yanto	4	if

Sorting Mahasiswa 2511531017

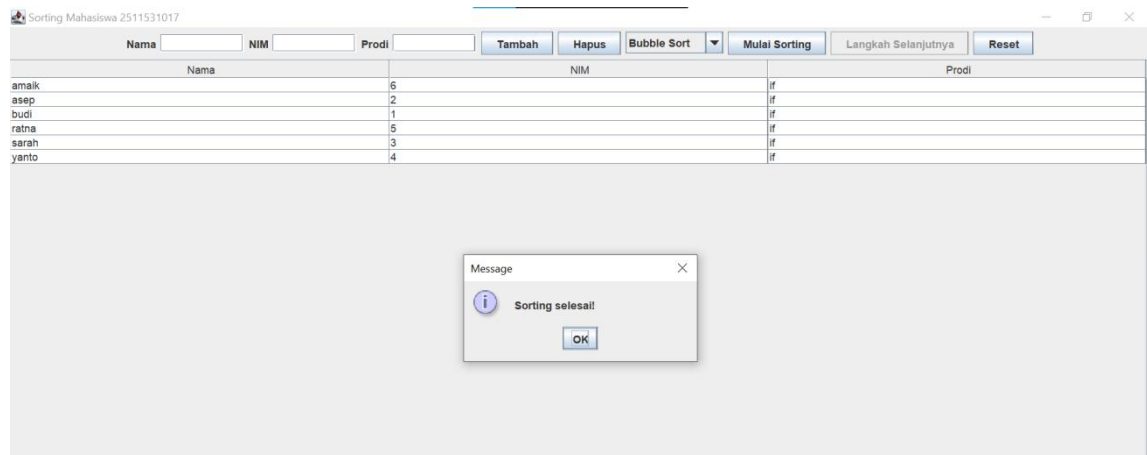
Nama NIM Prodi

Nama	NIM	Prodi
asep	2	if
amaik	6	if
budi	1	if
ratna	5	if
sarah	3	if
yanto	4	if

Sorting Mahasiswa 2511531017

Nama NIM Prodi

Nama	NIM	Prodi
amaik	6	if
asep	2	if
budi	1	if
ratna	5	if
sarah	3	if
yanto	4	if



Gambar 2.6

BAB III

KESIMPULAN

3.1 Kesimpulan

Algoritma sorting seperti Bubble Sort, Selection Sort, dan Insertion Sort digunakan untuk mengurutkan data berdasarkan nilai tertentu secara sistematis. Setiap algoritma memiliki cara kerja yang berbeda dalam melakukan perbandingan dan pertukaran data hingga seluruh data menjadi terurut. Perbedaan ini memengaruhi efisiensi dan langkah-langkah yang dilakukan pada proses pengurutan. Oleh karena itu, pemilihan algoritma yang tepat sangat penting dalam pengolahan data.

Pengurutan data dapat diimplementasikan menggunakan bahasa pemrograman Java, baik dalam bentuk program sederhana maupun GUI. Proses sorting juga dapat ditampilkan secara langkah demi langkah sehingga lebih mudah dipahami dalam proses analisis algoritma. Dengan adanya visualisasi, pengguna dapat melihat perubahan data secara langsung setiap tahap pengurutan. Hal ini membuat proses pembelajaran algoritma sorting menjadi lebih interaktif dan mudah dimengerti.

3.2 Saran

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

DAFTAR PUSTAKA

- [1] Oracle, “Arrays (Java Platform SE) – Sorting Algorithms Concepts,” Oracle Documentation, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/collections/interfaces/list.html>
- [2] Oracle, “Java Swing Tutorial,” Oracle Docs, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing>